

Spec-Driven Development with Agentic AI – Course Outline

Duration

24 Hours (3 Days).

Objectives

At the end of this workshop, participants will be able to:

- Explain the difference between prompt-driven (“vibe coding”) and spec-driven development, and the failure modes SDD addresses.
- Place the spec-driven lifecycle within the team's existing SDLC, requirements, design, build, test, release, and maintenance, rather than treating it as a separate process.
- Systematically build understanding of an existing, large, multi-service codebase using MCP-connected design documentation and code, before writing a spec against it.
- Establish project or per-service governance with a constitution, and use it to keep specs, plans, and generated code consistent.
- Write clear, testable specifications from both new-feature requests and existing bug or defect reports, using structured requirement formats (e.g., EARS-style statements), and clarify them before planning begins.
- Use GitHub Copilot and/or Claude Code, working from a shared VS Code foundation, to run the same spec-driven workflow regardless of which tool a team has access to.
- Run the full spec-driven lifecycle (constitution, specify, clarify, plan, tasks, checklist, analyze, implement) using the GitHub Spec Kit CLI, and recognize how OpenSpec structures the same workflow differently.
- Decompose a technical plan into small, agent-executable, independently verifiable tasks.
- Use checklist and analyze gates, including token/cost budgeting and security, compliance, and code-coverage checks, to catch issues before code is written.
- Apply human-in-the-loop review to agent-generated code, recognizing hallucinations, unsafe changes, and weak assumptions that automated gates miss, with added scrutiny for production systems.
- Maintain specs, constitutions, and agent configuration/instruction files as living documentation that stays audit-ready and in sync with the codebase.
- Delegate focused sub-tasks to a second agent instance, and know which tasks still require direct engineer control.
- Apply spec-driven development within a team workflow: repo structure, issue/PR linkage, CI integration, and governance.
- Form a project team on Day 1 and evolve one real project through the full spec-driven lifecycle across all three days, rather than working disconnected exercises.

Audience

Software engineers who are already using AI coding assistants in their day-to-day work and want to formalize that into a repeatable, spec-driven practice, including applying it across a large, established codebase alongside their team's existing toolchain.

Pre-requisite

- Working knowledge of Git and a standard development workflow (branches, PRs, code review).
- Regular hands-on use of at least one AI coding assistant (e.g., GitHub Copilot or Claude Code).

- Basic familiarity with VS Code and the command line.
- No prior spec-driven development experience is required.

Hardware & Network Requirements

- Laptop/desktop with minimum 16 GB RAM.
- Open internet connection (required for agent and MCP connectivity).
- Local admin access (to install CLI tools, agent extensions, and MCP server components).

Software Requirements

- VS Code with the GitHub Copilot extension (licensed and provisioned) and/or the Claude Code CLI, installed and authenticated ahead of the session, depending on team access.
- GitHub Spec Kit CLI (specify).
- OpenSpec CLI, for the Module-1 framework comparison.
- Git client and access to one shared, purpose-built, multi-service sample project repository, designed to mirror the complexity of a real system and containing multiple requirements and known defects, one assigned per team, provided ahead of the session.

Outline

Day 1

Module-1: From Vibe Coding to Spec-Driven Development

- Failure modes of ad hoc AI prompting: context drift, hallucinated APIs, unreviewable diffs, silent scope creep.
- Spec-driven development (SDD) principles: the spec as source artifact, code as build output; landscape of SDD frameworks (GitHub Spec Kit, OpenSpec, Kiro, BMAD-METHOD, Tessel) and why this course uses Spec Kit as the hands-on framework for its vendor-neutral, CLI-first design, with a quick side-by-side look at OpenSpec later today.
- The full SDD lifecycle: constitution → specify → clarify → plan → tasks → checklist → analyze → implement, and how each stage catches a different class of failure; this is the map for the rest of this course.

Hands-on:

- Form your project team, receive your team's assigned requirement (or defect) within the shared project repository, and set up your agentic coding tool (GitHub Copilot or Claude Code) in VS Code against it; GitHub Copilot and Claude Code cover the same capabilities with different commands, see the separate Tool Reference handout.
- Install the GitHub Spec Kit CLI (specify) and scaffold a specs/ directory for your team's project.
- Also initialize an openspec/ directory with the OpenSpec CLI, and compare the two scaffolds side by side to see how each framework structures the same spec-driven workflow.
- Run a basic and an advanced prompt on a small piece of your project to compare direct prompting versus structured, spec-driven input.

Module-2: Discovery at Scale, Understanding an Existing Hierarchical Codebase

- Why reading code alone doesn't scale across a large, hierarchical, multi-service codebase, and what to gather before writing a single spec.
- Connecting an agent to existing design documentation (wikis, architecture docs, prior specs) and to the team's bug/defect tracker (e.g., Jira, Azure DevOps, GitHub Issues) through MCP, alongside the code itself, to reconstruct context and surface known issues.
- Scoping discovery to a service boundary rather than the whole repository, and building a lightweight service/dependency map as a precursor artifact.

Hands-on:

- Connect an MCP server or connector to your project's design documentation and to its bug/defect tracker.
- Have the agent reconstruct your project's architecture and responsibilities from documentation plus code, and pull in one open defect against your project.
- Produce a one-page service/dependency map for your project, noting the defect you pulled in, to carry forward through the rest of the course.

Module-3: Establishing Governance and Writing a Clarified Specification

- Constitution: encoding project or per-service principles, code quality standards, testing requirements, and architectural constraints, informed by Module-2's discovery.
- Structuring requirements using EARS-style statements (“when/if... the system shall...”); writing a spec from a new-feature request as well as from an existing bug or defect pulled straight from the tracker; writing testable, unambiguous acceptance criteria.
- Clarify: running a dedicated pass to surface ambiguities and edge cases in a spec before it moves to planning.
- Where this fits inside the SDLC you already run: the constitution and a clarified spec align to requirements and design, the plan and tasks you'll write next align to technical design and iteration planning, checklist and analyze align to QA and test planning, implement aligns to development, and review plus living documentation align to code review, QA, and maintenance.

Hands-on:

- Write a constitution for your project, building on what your team discovered in Module-2 and encoding its existing conventions rather than inventing new ones.
- Draft two specs against that constitution: one for your project's next feature, one for the defect your team pulled from the tracker in Module-2. Run a clarify pass on both and resolve the ambiguities each surfaces.
- As a team, choose one spec to carry forward through the rest of the course; set the other aside, you'll return to it in Module-8.

Day 2

Module-4: From Clarified Spec to Reviewable Plan

- Plan: translating a clarified, constitution-aligned spec into a technical plan, covering architecture choices, constraints, and non-functional requirements.
- Using a plan/preview mode to produce a reviewable plan before any file changes are made.
- Task decomposition: breaking the plan into small, independently verifiable, agent-executable tasks.

Hands-on:

- Generate a plan from your project's spec (written and clarified on Day 1) and review it line by line as a team before accepting it.
- Edit the plan to correct at least one architecture or scope decision your team disagrees with.
- Decompose the approved plan into tasks and confirm each task maps back to a specific requirement.

Module-5: Quality Gates Before Implementation

- Checklist: generating domain-specific quality checklists (a “unit test suite for requirements”) to validate completeness, clarity, and consistency before code is written.
- Analyze: cross-checking the spec, plan, and tasks against each other and against the constitution to catch drift before implementation starts.
- Budgeting agent token usage and cost across a plan, and where security, compliance, and code-coverage checks fit as additional gates.

Hands-on:

- Generate and run a checklist against your project's spec; fix at least one gap it surfaces.

- Run an analyze pass across your project's spec, plan, and tasks; resolve any inconsistency it flags.
- Add a security/compliance or code-coverage checklist item, then implement one task in your project end-to-end only after it clears all gates.

Module-6: Reviewing, Validating, and Maintaining Agent Output

- Human-in-the-loop checkpoints: what to verify before accepting implemented code, even after it has cleared the checklist and analyze gates, with added scrutiny for production systems.
- Recognizing hallucinations, unsafe changes, and weak assumptions that automated gates won't catch.
- Managing spec drift: keeping the spec, constitution, and agent instruction files in sync with the codebase as it evolves, so they remain accurate as an audit trail.

Hands-on:

- Review a set of intentionally flawed agent-generated changes (instructor-provided) against their source spec, to practice spotting planted issues.
- Identify and document what a human reviewer, or which upstream gate, should have caught before merging.
- Apply the same review lens to your own project's Module-5 implementation, and update your project's spec, constitution, or instruction files to close any gap you find.

Day 3

Module-7: Multi-Agent Workflows & Team Adoption

- Coordinating multiple agents/subagents across a task graph; when to parallelize vs. sequence work.
- Repository conventions: a specs/ subdirectory per service, linking specs to issues and pull requests.
- Governance: reviewing agent actions before execution, managing secrets and permissions, and keeping an audit trail across teams and toolchains.

Hands-on:

- Propose and review an agent-generated plan for the next multi-step increment of your project before execution.
- Delegate a focused sub-task on your project to a second agent instance.
- Critique the delegated output and document what review step should have caught any issues.

Module-8: Common Pitfalls & Anti-Patterns

- Over-specifying vs. under-specifying: finding the right level of detail, and why a thin constitution or a skipped clarify pass usually shows up here first.
- Token and cost tradeoffs: budgeting more tokens per feature, typically offset by far fewer rework cycles.
- Case studies: spec-driven workflows that succeeded and failed at scale, and why.

Hands-on:

- Critique the spec your team set aside on Day 1 for over-/under-specification issues, now that you have a few days of hindsight from building against its counterpart.
- Rewrite that spec and re-run it through your agentic coding tool to compare output quality before and after.

Capstone Project

- Take the spec your team set aside on Day 1, and rewrote in Module-8, through the full lifecycle for the first time: plan, tasks, checklist, analyze, and implement, applying everything practiced on your other increment across the week, now with less guidance.
- Documented review trail: record where issues were caught (or missed) at each gate across both increments over the whole week, referencing the practices from Modules 2–6.

- Team presentation of the project's evolution from Day 1 through both increments, with peer and instructor feedback, followed by course wrap-up and certificate of completion.